

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Containers with Podman and Docker

Unit 14 • Linux (10-151-105)



Learning Outcomes

- Explain what a container is and how it differs from a virtual machine.
- Distinguish an image, a container, and a registry, and pull images into a host.
- Run, inspect, and manage containers with Docker and Podman.
- Build a custom image from a Containerfile.
- Give containers persistent storage with volumes and network access with port mapping.
- Recognize, at an awareness level, what orchestration tools like Compose and Kubernetes do.

LESSON 14.1

Container Foundations and Operations



What Is a Container?

- A container is a complete, portable package of an application plus everything it needs to run — its code, runtime, libraries, and configuration.
- Because it is self-contained, it behaves the same anywhere a compatible engine is installed: laptop, on-prem server, or cloud host.
- Common examples: nginx and Apache web servers, MySQL and MongoDB databases, monitoring tools like Prometheus.
- **Hold three terms apart:**
 - Engine — the software that builds and runs containers (Docker, Podman).
 - Image — a read-only template defining what a container contains.
 - Container — a running instance created from an image.

Image, Container, Engine: the Mental Model

- Think of the image as an application installer, and the container as the installed, running program.
- Downloading an installer isn't the same as running the software — and having an image on disk isn't the same as having a running container.
- **You never run an image directly. You run a container FROM it.**
- One image, many containers: the same nginx image can back dozens of identical, independent containers.

Containers vs. Virtual Machines



Different kinds of virtualization for different problems — not better or worse.

	Virtual machine	Container
Virtualizes	Hardware (via a hypervisor)	The operating system
Operating system	Full guest OS in every VM	Shares the host kernel; no guest OS
Size & startup	Large; seconds to minutes	Small; starts in well under a second
Isolation	Strong — separate kernels	Lighter — shared kernel
Best at	Different OSES; strong separation	Packaging and scaling copies of an app

The Two Engines: Docker and Podman

- A container engine pulls images, builds them, and runs containers on a single OS.
- Docker — popularized containers; the common choice on Ubuntu. Runs a persistent root daemon (dockerd); your commands are requests to it.
- Podman — the default on Rocky and the Red Hat family. Daemonless: launches each container as a child process, and runs rootless (as your user) by default.
- **Good news for learning both: Podman is command-compatible with Docker. Same subcommands — pull, run, ps, build, stop, rm.**
- A Containerfile and a Dockerfile are the same file under two names.

Docker vs. Podman: the Real Differences



Same commands, different architecture and privilege model.

	Docker (Ubuntu)	Podman (Rocky)
Architecture	Persistent daemon (dockerd)	Daemonless — child processes
Default privilege	Daemon runs as root	Rootless — runs as your user
Command	docker	podman (same subcommands)
Build file	Dockerfile	Containerfile (Dockerfile too)
Security posture	Rootless available, not default	Rootless is the default

Images and Registries

- Images come from a registry — a centralized repository for storing and sharing images.
- Public registries (Docker Hub, and registries from Google, AWS, Microsoft) or private ones restricted to an organization.
- pull downloads an image to your host: `docker pull nginx`
- push uploads an image you've built to a registry.
- List local images with `docker images`. Remove one with `docker rmi {image}`.
- Tags pin a version — `nginx:1.27`. Omitting the tag assumes `:latest`, fine for testing but risky in production where you want an exact, repeatable version.

Running Containers: run = pull + create + start

- The long way is two steps: `docker create` builds a container without starting it; `docker start` runs it.
- **The short way you'll use almost every time: `docker run` does three jobs at once** —
 - pulls the image if it isn't present,
 - creates the container,
 - and starts it.
- `--name` gives a memorable name instead of a random one.
- `-it` (interactive + TTY) keeps it running with a shell attached: `docker run -it --name web alpine sh`

Managing Containers

Containers are ephemeral by design — create freely, prune often.

Command	What it does
start / stop / restart	Start a stopped container, stop a running one, or both
ps / ps -a	List running containers — or ALL, including stopped ones
logs	Show a container's output — first stop when an app misbehaves
inspect	Detailed configuration and status as JSON
exec	Run a command inside a running container (e.g. <code>exec -it web sh</code>)
rm / rmi	Remove a container, or remove an image
prune	Bulk-remove stopped containers / unused images to reclaim resources

Building Your Own Image

- Pulling a ready-made image is the common case; building your own gives you exactly the software you need.
- Describe the image in a Containerfile (Docker: Dockerfile) — a plain-text recipe of UPPERCASE instructions:
 - FROM — the base image; always first.
 - RUN — run a command during the build, usually to install software.
 - CMD — the default command when the container starts.
 - USER — the user the container runs as (use it to drop root).
 - ENTRYPOINT — the container's primary executable (stricter than CMD).

A Minimal Containerfile

A Containerfile starting from Alpine that installs the nano editor. Note Alpine uses the apk package manager — the dual-ecosystem thinking follows you into containers.

LIVE DEMO

Write a two-line Containerfile, build it, and run a container from the new image.

```
FROM alpine
RUN apk add nano
CMD ["sh"]

# Build it (mind the trailing dot — the build context):
docker build --tag mynano:1.0 .
```

LESSON 14.2

Storage, Networking, and Beyond



Containers Are Stateless

- By design, anything written inside a running container is lost when that container is destroyed.
- That's a feature: it's what lets you throw containers away and recreate them freely.
- **But real apps — a database, a file store — need data that survives.**
- The answer: keep the data OUTSIDE the container and connect it in.
- Two ways to do that: volumes and bind mounts.

Volumes vs. Bind Mounts



Keep data outside the container so it survives the container's lifecycle.

	Volumes	Bind mounts
Managed by	The container engine	The host operating system
Lives where	An engine-managed area on the host	Any host path you choose
Portability	More portable between hosts	Tied to a specific host path
Typical use	Default choice for app data	Sharing a specific host directory in

Using a Volume

Create a volume, then attach it so a path inside the container maps to persistent storage. Data written there survives the container being destroyed and recreated.

LIVE DEMO

Create a volume, run a container against it, write data, destroy and recreate — show the data persists.

```
docker volume create appdata
docker run -v appdata:/var/lib/data nginx

# Rocky/SELinux: relabel the mount so the container may use it
docker run -v /host/path:/data:Z nginx

docker volume prune    # clear unused volumes
```

Container Networking

- Almost every useful container needs the network — users connect in, or the app reaches out.
- The engine gives containers virtual networks, attached to the host's real network by a network driver (like a virtual switch):
 - bridge — default; standalone containers on one host.
 - host — container shares the host's network stack directly.
 - overlay — containers across different hosts.
 - none — networking disabled, fully isolated.
- Manage with `docker network` (`create`, `ls`, `prune`); attach with `--network {name}`.

Port Mapping: Publishing a Port

A service inside a container isn't reachable from outside unless you connect its port to a host port. The `-p` flag does that, in host:container order.

LIVE DEMO

Publish nginx on port 8080 and reach the containerized web server from the host.

```
docker run -p 8080:80 nginx
```

```
# host port 8080 -> container port 80 (where nginx serves)
```

```
# browse to the host's port 8080 to reach the container
```

Container Security: a Shared Kernel Means Care



Security Focus

- Prefer rootless — Podman runs as your user by default, so a container process has only your privileges, not the system's.
- Trust your base images — an image is software you're about to run; pull from official or verified sources (the Unit 6 trusted-source discipline).
- Avoid privileged containers — flags like `--privileged` hand broad host access; use them only with a specific reason.
- Publish only the ports you need — each `-p` mapping is a door into the container from the network.
- Drop privilege inside the image too — a Containerfile `USER` line keeps the app from running as root.

Why it matters: containers make it trivially easy to run someone else's software in seconds — which is exactly why the security choices happen up front, at the image and the run command, not afterward.

Orchestration at a Glance (Awareness)

- Everything so far runs containers by hand, a few at a time — exactly the skill this unit builds.
- In production, apps grow to dozens or hundreds of containers across many hosts. Orchestration automates managing them at scale.
- Docker Compose — describes a multi-container app on ONE host in a `compose.yaml`; brings it all up with `docker compose up`.
- Docker Swarm — Docker's own clustering across multiple hosts; simpler than Kubernetes, less common today.
- Kubernetes (K8s) — the industry standard for large, cloud-native systems; manages pods, declared via deployments, exposed by services.
- **Awareness-level only — recognize the tools; you won't build a cluster in this course.**

What Orchestration Looks Like (Awareness)

A Compose file reads much like the run options you already know, just written down — a web server and a database brought up together:

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
  db:
    image: mysql
    volumes:
      - dbdata:/var/lib/mysql
volumes:
  dbdata:
```

This Week's Lab

NETLAB+ Lab: Containers with Podman and Docker

Pull and run images, list and inspect containers, map a published port, mount a volume, and build a small custom image from a Containerfile — using **Docker on Ubuntu** and **rootless Podman on Rocky**, side by side.

Starting point: snapshot WK14_CONTAINERS, both VMs. Work does not persist between reservations — capture your evidence before logging out.

Graded on the observable end state on each host — images pulled, containers running, a port published, a volume mounted, and a working custom image — not on a written description.

Key Takeaways

- A container packages an app with everything it needs; it shares the host kernel, so it's lighter than a VM — and a root container is effectively root on the host.
- Image = read-only template; container = running instance; registry = where images live.
- Docker (Ubuntu) and Podman (Rocky) share nearly the same commands; Podman is daemonless and rootless by default.
- run = pull + create + start. Use `ps -a` to see stopped containers. Build images from a Containerfile.
- Volumes give persistence; `-p host:container` publishes a port. Open only what you need.
- Orchestration (Compose, Kubernetes) manages containers at scale — awareness-level here.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 14 · Containers with Podman and Docker

